Universidad
Nacional de General Sarmiento

Profesor: Lucas Bidart

Programacion I

Segundo Semestre 2026 Trabajo Practico: Delivery Rush



Integrantes del Grupo

Nombre	Apellido	N.º Legajo	Emai
Franco	Schermer	45306862	francoschermer@gmail.com
Jeremias	Vazquez	45430163	jeremiasalejovazquez@gmail.com
Belen	Rebeco	36567151	belenrebeco@hotmail.com

Introducción:

El presente trabajo práctico consiste en el desarrollo de un videojuego llamado **Delivery Rush**, programado en Java utilizando la librería Entorno, proporcionada por los docentes de la universidad. La temática del juego consiste en simular el trabajo de un repartidor que debe entregar una determinada cantidad de pedidos, mientras enfrenta enemigos que intentan impedir su tarea y eliminarlo mediante el uso de bombas.

Para implementar el código, se utilizaron conceptos vistos en la materia **Programación 1**, como la definición de clases, atributos, constructores y métodos. Se organizaron las distintas clases del juego, como Repartidor, Proyectil y Mapa, dentro del mismo paquete llamado juego. Además, en la clase principal Juego se definieron los métodos necesarios para coordinar el funcionamiento de los distintos objetos y controlar sus movimientos.

Como los elementos del juego deben actualizar sus posiciones y estados constantemente, se utiliza el método tick () como parte del ciclo principal de ejecución. Este permite actualizar la pantalla y reflejar los cambios que se producen durante la partida.

A lo largo de este informe, se explicará cómo se relacionan las distintas clases, qué dificultades técnicas surgieron durante el desarrollo y qué soluciones se implementaron para lograr el correcto funcionamiento de la clase principal Juego y del sistema en general.

Descripción:

Clase Mapa

La clase Mapa se encarga de crear y organizar las cuadras que forman parte del mapa del juego. Utiliza un arreglo de 16 cuadras, distribuidas en cuatro filas y cuatro columnas. También se ocupa de dibujarlas en la pantalla y de proporcionar acceso al arreglo para que otras clases puedan utilizarlo.

Variables de instancia

- cuadras: es un arreglo de tipo Cuadra que almacena las 16 cuadras que forman el mapa.

Métodos

- Mapa (): es el constructor de la clase. Crea un arreglo para almacenar 16 cuadras y utiliza dos ciclos for para recorrer las cuatro filas y las cuatro columnas. En cada repetición calcula las coordenadas de una cuadra y la crea con un ancho de 100 y un alto de 90. Las coordenadas se calculan a partir de valores iniciales y separaciones fijas para distribuir las cuadras en el mapa.
- dibujar(Entorno entorno): recorre el arreglo de cuadras y dibuja cada una que no sea null, utilizando el método dibujar() de la clase Cuadra.
- getCuadras(): devuelve el arreglo de cuadras para que otras clases, como Juego, puedan acceder a ellas y utilizar la información de las casas y los negocios que contienen.

Decisiones de diseño

La creación de la clase Mapa fue fundamental para comenzar a construir el escenario del juego, ya que necesitábamos organizar la distribución de las calles y las cuadras antes de incorporar los demás elementos. Según los requerimientos del TP, el mapa debía tener una grilla de al menos cuatro filas por cuatro columnas, y cada cuadra debía contener como mínimo tres casas y un negocio.

Para cumplir con estos requisitos, creamos un arreglo de 16 cuadras y utilizamos dos ciclos for para distribuirlas en cuatro filas y cuatro columnas. También establecimos valores iniciales para las coordenadas y una separación fija entre las cuadras, de manera que quedaran organizadas dentro del espacio disponible en la pantalla.

La creación de las casas y los negocios se delegó en la clase Cuadra, que genera tres casas y un negocio cada vez que se crea una cuadra. De esta manera, organizamos el código separando la construcción general del mapa de los elementos que contiene cada cuadra.

Problemas encontrados y soluciones

Una de las dificultades fue definir las coordenadas y la separación entre las cuadras para que quedaran distribuidas de manera ordenada dentro de la pantalla. Para resolverlo, establecimos valores iniciales y una separación fija entre las filas y las columnas. Así logramos una distribución regular que cumple con los requisitos del TP.

Código de implementación

```
package juego;

import entorno.Entorno;

public class Mapa {
private Cuadra[] cuadras;

public Mapa() {
this.cuadras = new Cuadra[16];
int index = 0;
for (int i = 0; i < 4; i++) {
for (int j = 0; j < 4; j++) {
int x = 130 + j * 180;
int y = 80 + i * 140;
this.cuadras[index] = new Cuadra(x, y, 100, 90);
index++;
}
}
}

public void dibujar(Entorno entorno) {
for (int i = 0; i < this.cuadras.length; i++) {
if (this.cuadras[i] != null) {
this.cuadras[i].dibujar(entorno);
}
}
}

public Cuadra[] getCuadras() {
return this.cuadras;
}
}
```

Clase Cuadra

La clase Cuadra representa cada bloque del mapa del juego. Se encarga de guardar su posición y sus dimensiones, crear las casas y el negocio que se encuentran dentro de ella, y dibujar estos elementos en la pantalla.

Variables de instancia

- x: guarda la coordenada horizontal de la cuadra.
- y: guarda la coordenada vertical de la cuadra.
- ancho: indica el ancho de la cuadra.
- alto: indica la altura de la cuadra.
- casas: es un arreglo que almacena las tres casas ubicadas dentro de la cuadra.
- negocio: guarda el negocio que pertenece a esa cuadra.

Métodos

- Cuadra (int x, int y, double ancho, double alto): es el constructor de la clase. Recibe la posición y las dimensiones de la cuadra, guarda esos valores y crea tres casas y un negocio. Las posiciones de estos elementos se calculan tomando como referencia las coordenadas de la cuadra, sumando o restando valores para ubicarlos en diferentes lugares.
- dibujar(Entorno entorno): dibuja el rectángulo verde que representa la cuadra. Luego recorre el arreglo y dibuja las casas, y finalmente dibuja el negocio. Antes de dibujar cada elemento, comprueba que no sea null, es decir, que exista un objeto en esa posición.
- getX(): devuelve la coordenada horizontal de la cuadra.
- getY(): devuelve la coordenada vertical de la cuadra.
- getAncho(): devuelve el ancho de la cuadra.
- getAlto(): devuelve la altura de la cuadra.
- getNegocio(): devuelve el objeto Negocio que pertenece a la cuadra.
- getCasas(): devuelve el arreglo que contiene las casas de la cuadra.

Decisiones de diseño

Decidimos que cada cuadra tuviera tres casas y un negocio, ubicados en distintas posiciones dentro de ella. Para organizar estos elementos, tomamos como referencia las coordenadas de la propia cuadra y utilizamos valores que se suman o se restan para establecer sus ubicaciones.

También organizamos la creación de las casas y los negocios dentro del constructor de Cuadra. De esta manera, cuando la clase Mapa crea las cuadras, cada una genera automáticamente sus propias casas y su negocio, sin necesidad de crearlos individualmente desde la clase Juego.

Problemas encontrados y soluciones

Una de las dificultades fue definir la ubicación de las casas y los negocios dentro de cada cuadra. Para resolverlo, fuimos probando distintas posiciones, sumando o restando valores a las coordenadas hasta encontrar una distribución que nos gustara a todos. De esta manera, elegimos las posiciones y dimensiones que consideramos más adecuadas para que los elementos quedaran ordenados dentro de las cuadras y el mapa tuviera una apariencia que nos convenciera como grupo.

Código de implementación

```
package juego;

import entorno.Entorno;

public class Cuadra {
private int x;
private int y;
private double ancho;
private double alto;
private Casa[] casas;
private Negocio negocio;

public Cuadra(int x, int y, double ancho, double alto) {
this.x = x;
this.y = y;
this.ancho = ancho;
this.alto = alto;
this.casas = new Casa[3];
this.casas[0] = new Casa(x - 20, y - 20, 22, 22);
this.casas[1] = new Casa(x + 20, y - 20, 22, 22);
this.casas[2] = new Casa(x - 20, y + 20, 22, 22);
```

```
this.negocio = new Negocio(x + 20, y + 20, 22, 22);
}

public void dibujar(Entorno entorno) {
entorno.dibujarRectangulo(x, y, ancho, alto, 0, java.awt.Color.green);
for (int i = 0; i < this.casas.length; i++) {
if (this.casas[i] != null) {
this.casas[i].dibujar(entorno);
}
}
if (this.negocio != null) {
this.negocio.dibujar(entorno);
}
}

public int getX() {
return x;
}

public int getY() {
return y;
}

public double getAncho() {
return ancho;
}

public double getAlto() {
return alto;
}

public Negocio getNegocio() {
return this.negocio;
}
public Casa[] getCasas() {
return this.casas;
}
}
```

Clase Casa

La clase Casa representa las casas del mapa donde el repartidor debe realizar las entregas. Se encarga de guardar su posición y dimensiones, dibujarlas en pantalla y comprobar si el mouse se encuentra sobre ellas.

Variables de instancia

- x e y: indican la posición de la casa en el mapa.
- alto y ancho: representan sus dimensiones.

Métodos

- Casa(...): inicializa la posición y las dimensiones de la casa.
- Métodos get: permiten consultar su posición, alto y ancho.
- dibujar(Entorno entorno): dibuja la casa como un rectángulo de color naranja.
- estaAdentro(double px, double py): comprueba si las coordenadas del mouse están dentro de los límites de la casa. Devuelve true si está dentro y false si está fuera.

Decisiones de diseño

Definimos posiciones y dimensiones para ubicar las casas dentro de las cuadras del mapa. Las casas mantienen esas posiciones durante el juego, mientras que la clase Juego selecciona aleatoriamente cuál será el destino de cada pedido.

También incorporamos el método estaAdentro() en la clase Casa, ya que esta contiene la posición y las dimensiones necesarias para comprobar si el mouse está dentro de sus límites. La clase Juego utiliza este método y verifica que el repartidor esté lo suficientemente cerca para completar la entrega. De esta manera, cada clase tiene una responsabilidad diferente y el código queda más organizado.

Problemas encontrados y soluciones

Una de las principales dificultades fue lograr que el mouse detectara correctamente cuándo estaba sobre una casa. Al principio, el área de detección era demasiado pequeña, por lo que el repartidor tenía que acercarse mucho para realizar la entrega. Esto complicaba el juego y hacía que se perdiera tiempo. Para solucionarlo, ajustamos el área de detección para facilitar la interacción y hacer que las entregas fueran más cómodas para el jugador.

Codigo implementado

```
package juego;

import java.awt.Color;

import entorno.Entorno;

public class Casa {

private int x;

private int y;

private double alto;

private double ancho;

     public Casa(int x, int y, double alto, double ancho) {

this.x = x;

this.y = y;

this.alto = alto;

this.ancho = ancho;

}

public int getX() {

return x;

}

public int getY() {

return y;

}
```

```
public double getAlto() {

return alto;

}

public double getAncho() {

            return ancho;

}

public void dibujar(Entorno entorno) {

entorno.dibujarRectangulo(x, y, ancho, alto, 0, Color.orange);

}

//Metodo para saber si el mouse esta dentro de las coordenadas de CASA

public boolean estaAdentro(double px, double py) {

    return px >= (this.x - this.ancho / 2) && px <= (this.x + this.ancho / 2)
&&

           py >= (this.y - this.alto / 2) && py <= (this.y + this.alto / 2);

}

}
```

Clase Repartidor

La clase Repartidor representa al personaje que controla el jugador durante la partida. Su función es guardar los datos del personaje, permitir su movimiento y mostrarlo en pantalla.

Variables de instancia

- x e y: guardan la posición del repartidor en el mapa.
- ancho y alto: definen las dimensiones del triángulo que lo representa.
- velocidad: determina la distancia que se desplaza en cada movimiento.
- ángulo: indica la orientación del repartidor y permite que el triángulo apunte hacia la dirección en la que se mueve.

Métodos de la clase

- Repartidor(): es el constructor. Inicializa la posición, las dimensiones y la velocidad con los valores recibidos, y establece la orientación inicial hacia arriba.
- dibujar(Entorno entorno): dibuja al repartidor en pantalla mediante un triángulo de color celeste, utilizando su posición, sus dimensiones y su orientación.
- moverDerecha(): aumenta la coordenada x según la velocidad y orienta al repartidor hacia la derecha.
- moverIzquierda(): disminuye la coordenada x según la velocidad y orienta al repartidor hacia la izquierda.
- moverArriba(): disminuye la coordenada y según la velocidad y orienta al repartidor hacia arriba.
- moverAbajo(): aumenta la coordenada y según la velocidad y orienta al repartidor hacia abajo.

- getX() y getY(): permiten consultar la posición horizontal y vertical del repartidor.
- getAncho() y getAlto(): permiten consultar sus dimensiones.
- getAngulo(): permite consultar su orientación actual.

Decisiones de diseño

Para diseñar la clase Repartidor, tomamos como referencia una clase de ejemplo que mostró el profesor en el aula, en la que un rectángulo podía desplazarse hacia la derecha y hacia la izquierda. A partir de ese ejemplo, adaptamos el código para representar al repartidor mediante un triángulo y agregamos los movimientos hacia arriba y hacia abajo, necesarios para que pudiera desplazarse en las cuatro direcciones.

También tuvimos en cuenta las dimensiones de la ventana del juego, de 800 por 600 píxeles, para establecer la posición inicial del repartidor y controlar sus movimientos dentro del mapa.

Por otro lado, incorporamos los métodos getters para que la clase Juego pudiera consultar la posición, las dimensiones y la orientación del repartidor. Las variables de instancia se declararon como private para evitar que otras clases las modifiquen directamente y prevenir cambios que puedan generar errores en el funcionamiento del juego. De esta manera, cada clase mantiene el control sobre sus propios datos, mientras que las demás pueden consultar la información necesaria mediante los métodos correspondientes.

Problemas encontrados y soluciones

Uno de los problemas que encontramos fue la orientación del repartidor durante sus movimientos. En el ejemplo que había mostrado el profesor en clase, el rectángulo solamente se desplazaba hacia la derecha y hacia la izquierda, por lo que el ángulo se mantenía en cero. Al adaptar ese código a nuestro juego, nos encontramos con que el repartidor también debía moverse hacia arriba y hacia abajo. Al principio, el triángulo se desplazaba, pero no cambiaba su orientación según la dirección del movimiento.

Para resolverlo, tuvimos que revisar cómo se representan los ángulos y los ejes de coordenadas en la pantalla. Observamos que el eje vertical funciona al revés de lo que solemos representar en matemática: los valores de y aumentan hacia abajo y disminuyen hacia arriba. A partir de esto, asignamos un ángulo diferente a cada dirección de movimiento: 0 para la derecha, Math.PI para la izquierda, Math.PI / 2 para abajo y 3 * Math.PI / 2 para arriba.

De esta manera, logramos que el triángulo cambiara su orientación de acuerdo con la dirección en la que se desplazaba el repartidor, solucionando el problema inicial.

Otro problema que detectamos durante las pruebas fue que el repartidor podía atravesar las cuadras del mapa. Para solucionarlo, incorporamos un sistema de detección de colisiones que comprueba si la posición a la que se quiere desplazar el repartidor se superpone con alguna cuadra. Si se detecta una colisión, se impide el movimiento en esa dirección. Así, logramos evitar que el repartidor atraviese los edificios.

Código de implementación

```
package juego;

import java.awt.Color;

import entorno.Entorno;

public class Repartidor {

    private double x;

    private double y;

    private double ancho;

    private double alto;

    private double velocidad;

    private double angulo;


    public Repartidor(double x, double y, double ancho, double alto, double
velocidad) {
```

```
        this.x = x;

        this.y = y;

        this.ancho = ancho;

        this.alto = alto;

        this.velocidad = velocidad;

        this.angulo = 3 * Math.PI / 2;

    }

    public void dibujar(Entorno entorno) {

        entorno.dibujarTriangulo(this.x,  this.y,  (int)  this.ancho,  (int)
this.alto, this.angulo, Color.CYAN);

    }

    public void moverDerecha() {

        this.x += this.velocidad;

        this.angulo = 0;

    }

    public void moverIzquierda() {

        this.x -= this.velocidad;

        this.angulo = Math.PI;

    }

    public void moverAbajo() {

        this.y += this.velocidad;

        this.angulo = Math.PI / 2;

    }



    public void moverArriba() {

        this.y -= this.velocidad;

        this.angulo = 3 * Math.PI / 2;
```

```
    }
    public double getX() {
        return x;
    }
    public double getY() {
        return y;
    }
    public double getAncho() {
        return this.ancho;
    }
    public double getAlto() {
        return this.alto;
    }
    public double getAngulo() {
        return this.angulo;
    }
    public void respawnRep(double x, double y) {
      this.x = x;
      this.y = y;
    }
    public Proyectil disparar() {
        return new Proyectil(this.x, this.y, this.angulo);
    }
}
```

Clase Proyectil

La clase Proyectil representa los disparos que realiza el repartidor durante la partida. Su función es guardar los datos del proyectil, permitir su desplazamiento y dibujarlo en pantalla.

Variables de instancia

- x e y: guardan la posición horizontal y vertical del proyectil en la pantalla.
- ancho y alto: definen las dimensiones del proyectil.
- velocidad: determina la rapidez con la que se desplaza el proyectil.
- ángulo: indica la dirección en la que se mueve el disparo.

Métodos de la clase

- Proyectil (): es el constructor. Inicializa la posición y el ángulo recibidos como parámetros, y establece las dimensiones en 8 y la velocidad en 5.0.
- mover (): actualiza la posición del proyectil según su velocidad y su ángulo, calculando el desplazamiento horizontal y vertical.
- dibujar (Entorno entorno): dibuja el proyectil en pantalla mediante un círculo de color magenta.
- getX() y getY(): permiten consultar la posición horizontal y vertical del proyectil.
- getAncho() y getAlto(): permiten consultar sus dimensiones, información que puede utilizarse desde otras clases para comprobar posibles colisiones.

Decisiones de diseño y dificultades encontradas

Al principio, el método mover () utilizaba una serie de condiciones (if-else) para determinar el movimiento según la dirección indicada: arriba, abajo, izquierda o derecha. Esto limitaba los

disparos a esas cuatro direcciones y no permitía que siguieran con precisión la orientación del repartidor.

```
public void mover () {
        if (this.direccion.equals("ARRIBA")) {
            this.y -= this.velocidad;
        } else if (this.direccion.equals("ABAJO")) {
            this.y += this.velocidad;
        } else if (this.direccion.equals("IZQUIERDA")) {
            this.x -= this.velocidad;
        } else if (this.direccion.equals("DERECHA")) {
            this.x += this.velocidad;
```

Para solucionar este problema, decidimos modificar el método mover () para que utilizara el ángulo de orientación del repartidor, recibido al crear el proyectil. Para ello, incorporamos las funciones Math.cos() y Math.sin(), que permiten calcular el desplazamiento horizontal y vertical a partir de ese ángulo y la velocidad del proyectil.

De esta manera, logramos que los disparos se desplazaran en la dirección hacia la que apunta el repartidor, sin necesidad de establecer una condición para cada dirección. Esta modificación también permitió simplificar el código y adaptar el movimiento del proyectil a la orientación del personaje.

Código de implementación

```
package juego;
import java.awt.Color;
import entorno.Entorno;
public class Proyectil {
    private double x;
```

```
private double y;
private double ancho;
private double alto;
private double velocidad;
private double angulo;


public Proyectil(double x, double y, double angulo) {
    this.x = x;
    this.y = y;
    this.ancho = 8;
    this.alto = 8;
    this.velocidad = 5.0;
    this.angulo = angulo;
}
public void mover()
    this.x += this.velocidad * Math.cos(this.angulo);
    this.y += this.velocidad * Math.sin(this.angulo);
}
public void dibujar(Entorno entorno) {
    entorno.dibujarCirculo(this.x, this.y, this.ancho, Color.MAGENTA);
}
 public double getX() {
    return x;
}
public double getY() {
    return y;
}
```

```
    public double getAncho() {
        return ancho;
    }
    public double getAlto() {
        return alto;
    }
}
```

Clase Flecha

La clase Flecha representa la indicación visual que guía al repartidor hacia el lugar donde debe dirigirse durante cada etapa del pedido.

Variables de instancia

- x e y: indican la posición en la que se dibuja la flecha.
- visible: determina si la flecha debe mostrarse en pantalla.

Métodos

- Flecha (int x, int y): es el constructor. Inicializa la posición de la flecha y establece que inicialmente no sea visible.
- dibujar (Entorno entorno): dibuja la flecha solamente cuando la variable visible es verdadera. Está formada por un rectángulo que representa el cuerpo y un triángulo que representa la punta.
- mostrar (int x, int y): actualiza la posición de la flecha y la hace visible.
- ocultar (): hace que la flecha deje de mostrarse en pantalla.

Problemas encontrados y soluciones

Uno de los problemas fue construir la flecha uniendo un rectángulo con un triángulo, ya que al principio la punta quedaba orientada hacia la derecha. Para solucionarlo, ajustamos el ángulo del triángulo hasta lograr que apuntara hacia arriba.

También tuvimos que corregir su ubicación, porque inicialmente aparecía sobre el repartidor. Modificamos su funcionamiento para que señalara primero el negocio y, luego, la casa donde debía realizar la entrega, utilizando las coordenadas que le proporciona la clase Juego.

Por último, agregamos una espera de dos segundos al inicio para que la flecha permaneciera oculta antes de mostrar la ubicación del negocio y comenzar el pedido.

Código de implementación

```
package juego;

import java.awt.Color;

import entorno.Entorno;

public class Flecha {

    private int x;

    private int y;

    private boolean visible;

    public Flecha(int x, int y) {

        this.x = x;

        this.y = y;

        this.visible = false;

    }

    public void dibujar(Entorno entorno) {
```

```
        if (visible) {
            entorno.dibujarRectangulo(this.x, this.y + 6, 10, 18, 0,
Color.black);
            entorno.dibujarTriangulo(this.x, this.y - 8, 20, 18, 3*Math.PI / 2,
Color.black);
        }
    }
    public void mostrar (int x, int y) {
        this.x = x;
        this.y = y;
        this.visible = true;
    }
    public void ocultar() {
        this.visible = false;
    }

}
```

Clase Negocio

Descripción general

La clase Negocio representa los negocios que se encuentran dentro de las cuadras del mapa. Se encarga de guardar su posición y sus dimensiones, dibujarlos en la pantalla y comprobar si las coordenadas del mouse están dentro de sus límites. Esto último permite que el jugador pueda hacer clic sobre el negocio para comenzar un pedido.

Variables de instancia

- x: guarda la coordenada horizontal del negocio.
- y: guarda la coordenada vertical del negocio.
- alto: indica la altura del negocio.
- ancho: indica el ancho del negocio.

Métodos

- Negocio (int x, int y, double alto, double ancho): es el constructor de la clase. Recibe la posición y las dimensiones del negocio y guarda esos valores en sus variables de instancia.
- getX(): devuelve la coordenada horizontal del negocio.
- getY(): devuelve la coordenada vertical del negocio.
- getAlto(): devuelve la altura del negocio.
- getAncho(): devuelve el ancho del negocio.
- Dibujar (Entorno entorno): dibuja el negocio como un rectángulo rojo en la pantalla, utilizando las coordenadas y dimensiones guardadas.
- EstaAdentro (double px, double py): comprueba si las coordenadas recibidas están dentro de los límites del negocio. Para hacerlo, compara la posición recibida con los bordes del rectángulo y devuelve true si el punto está dentro, o false si está fuera.

Decisiones de diseño

Decidimos representar los negocios con rectángulos rojos para distinguirlos visualmente de las casas y las cuadras del mapa. La posición y las dimensiones de cada negocio se establecen cuando se crea la cuadra a la que pertenece.

También incorporamos el método estaAdentro() para comprobar si el mouse está dentro de los límites del negocio. Elegimos dejar esta comprobación en la clase Negocio porque esta clase conoce su propia posición y sus dimensiones. Luego, la clase Juego utiliza el método para verificar si el jugador hizo clic sobre el negocio, junto con la comprobación de que el repartidor esté suficientemente cerca.

Problemas encontrados y soluciones

Para permitir que el jugador pudiera seleccionar un negocio con el mouse, fue necesario comprobar si las coordenadas del clic estaban dentro de sus límites. Para resolverlo, implementamos el método estaAdentro(), que calcula los bordes del rectángulo a partir de su posición central, su ancho y su alto. De esta manera, la clase Juego puede utilizar esa comprobación para determinar si el clic se realizó sobre el negocio.

Código de implementación

```
package juego;

import java.awt.Color;

import entorno.Entorno;

public class Negocio {

private int x;

private int y;

private double alto;

private double ancho;

public Negocio(int x, int y, double alto, double ancho) {

this.x = x;
```

```
this.y = y;

this.alto = alto;

this.ancho = ancho;

}

public int getX() {

return x;

}

public int getY() {

return y;

}

public double getAlto() {

return alto;

}

public double getAncho() {

return ancho;

}

public void dibujar(Entorno entorno) {

entorno.dibujarRectangulo(x, y, ancho, alto, 0, Color.red);

}

//Metodo para saber si el mouse esta dentro de las coordenadas del NEGOCIO

public boolean estaAdentro(double px, double py) {

    return px >= (this.x - this.ancho / 2) && px <= (this.x + this.ancho / 2)
&&

           py >= (this.y - this.alto / 2) && py <= (this.y + this.alto / 2);

}

}
```

Clase Interfaz

La clase Interfaz se encarga de mostrar la información del juego en la parte superior de la pantalla. Muestra el tiempo transcurrido, las vidas del repartidor, los pedidos entregados, los pedidos perdidos, los enemigos eliminados y el estado del pedido actual. De esta manera, el jugador puede consultar la información principal mientras juega.

Variables de instancia

La clase no tiene variables de instancia propias. Los datos que necesita para mostrar la información los recibe como parámetros del método dibujar().

Métodos

- Dibujar (Entorno entorno, int vidas, int pedidosEntregados, int pedidosPerdidos, int enemigosEliminados, String estadoPedido): dibuja la barra superior y muestra los datos del juego. Recibe como parámetros el entorno, la cantidad de vidas, los pedidos entregados y perdidos, los enemigos eliminados y el estado del pedido actual.

- Primero, dibuja un rectángulo gris como fondo de la barra. Después, establece la fuente y el color del texto, calcula los segundos transcurridos dividiendo el tiempo del juego por 1000 y muestra cada dato en una posición determinada de la pantalla. Finalmente, muestra el estado del pedido con un tamaño de letra mayor para que se destaque.

Decisiones de diseño

Decidimos crear una clase independiente para la interfaz con el objetivo de separar la información que se muestra en pantalla de la lógica principal del juego. De esta manera, la clase Juego puede encargarse del funcionamiento del juego y llamar al método dibujar () de Interfaz para mostrar los datos.

También distribuimos la información en diferentes posiciones de la barra superior para que los datos principales pudieran verse durante la partida. Elegimos un fondo gris y utilizamos distintos tamaños de letra para organizar la información y destacar el estado del pedido.

Problemas encontrados y soluciones

A medida que fuimos avanzando con el desarrollo del juego, fuimos incorporando mejoras para mostrar con mayor claridad la situación del jugador. Una de ellas fue agregar la cantidad de pedidos perdidos a la interfaz, para que el jugador pudiera saber no solo cuántos pedidos había entregado, sino también cuántos no había logrado completar.

Además, fuimos modificando y probando los tiempos del juego para evaluar si eran adecuados para las distintas situaciones que podían presentarse. Tuvimos en cuenta el tiempo necesario para llegar al negocio, retirar el pedido y llevarlo hasta la casa, así como las posibilidades de ganar o perder. A partir de estas pruebas, fuimos ajustando los valores para buscar un equilibrio y lograr que los tiempos fueran razonables y permitieran jugar de manera adecuada.

Código de implementación

```
package juego;

 import entorno.Entorno;

import java.awt.Color;

public class Interfaz {

    public void dibujar(Entorno entorno, int vidas, int pedidosEntregados,int
pedidosPerdidos, int enemigosEliminados, String estadoPedido) {

        // 1. Fondo de la barra superior

        entorno.dibujarRectangulo(400, 20, 800, 40, 0, Color.GRAY);

        // 2. Textos principales en blanco

        entorno.cambiarFont("Arial", 14, Color.black);

        int segundos = entorno.tiempo() / 1000;

        entorno.escribirTexto("Tiempo: " + segundos + "s", 15, 25);

        entorno.escribirTexto("Vidas: " + vidas, 120, 25);

        entorno.escribirTexto("Entregados: " + pedidosEntregados, 220, 25);

        entorno.escribirTexto("Pedidos Perdidos: " + pedidosPerdidos,340 , 25);
```

```
        entorno.escribirTexto("Eliminados: " + enemigosEliminados, 500, 25);
        // 3. Estado del pedido en amarillo para destacarlo
        entorno.cambiarFont("Arial", 17, Color.white);
        entorno.escribirTexto("Pedido: " + estadoPedido, 630, 25);
    }
}
```

Clase Bomba

La clase Bomba se encarga de representar una bomba dentro del juego. Controla su cuenta regresiva, el momento en que explota, el crecimiento de la explosión y el tiempo durante el cual permanece activa. La explosión cambia de tamaño según la dirección que tenga asignada.

Variables de instancia

- x e y: indican la posición de la bomba en la pantalla.
- ancho y alto: representan las dimensiones de la bomba y de la explosión, que aumentan cuando esta se produce.
- exploto: indica si la bomba ya explotó. Inicialmente tiene el valor false.
- mechaBomb: controla la cuenta regresiva de la bomba. Comienza en 180 y disminuye hasta llegar a cero.
- direccion: indica la dirección de la explosión y determina cómo aumenta su ancho y su alto.
- velocidadImplosion: establece cuánto aumentan las dimensiones de la explosión en cada actualización. En este caso, el valor inicial es 2.
- tiempoDeExplosion: cuenta el tiempo que la explosión permanece activa.

Métodos

- Bomba(int x, int y, double ancho, double alto, int direccion): es el constructor. Inicializa la posición, las dimensiones y la dirección de la bomba. También establece que todavía no explotó, configura la cuenta regresiva en 180, define la velocidad de crecimiento en 2 y pone el contador de duración en cero.

- cuentaRegresiva(): disminuye el valor de mechaBomb mientras sea mayor que cero. Cuando llega a cero, cambia exploto a true para indicar que la bomba explotó.
- dibujar(Entorno entorno): dibuja la bomba en la pantalla. Mientras no explotó, aparece como un círculo gris oscuro. Cuando explota, se representa mediante un rectángulo amarillo.
- getXBomb(): devuelve la coordenada horizontal de la bomba.
- getYBomb(): devuelve la coordenada vertical de la bomba.
- getAltoBomb(): devuelve la altura actual de la bomba o de la explosión.
- getAnchoBomb(): devuelve el ancho actual de la bomba o de la explosión.
- crecerExplosion(): aumenta el tamaño de la explosión cuando la bomba ya explotó. Según la dirección, establece distintos límites para el ancho y el alto, lo que permite representar explosiones horizontales o verticales.
- duracionExp(): controla cuánto tiempo permanece activa la explosión. Mientras el contador sea menor que 120, lo incrementa y devuelve false. Cuando alcanza ese límite, reinicia el contador y devuelve true, indicando que terminó su duración. Si la bomba todavía no explotó, devuelve false.

Problemas encontrados y soluciones

Una de las principales dificultades fue definir en qué momento debía lanzar una bomba cada enemigo. Nos costó lograr que los lanzamientos se produjeran en distintos momentos, ya que no queríamos que todos los enemigos soltaran sus bombas al mismo tiempo.

Para solucionarlo, establecimos distintos tiempos de espera para cada enemigo, de manera que los lanzamientos se realizaran de forma escalonada. De esta manera, logramos controlar mejor cuándo aparecía cada bomba durante el juego.

Por otra parte, implementamos el funcionamiento de las bombas mediante una cuenta regresiva que determina cuándo explotan, un crecimiento de la explosión según su dirección y un tiempo de duración antes de que desaparezca.

Código de implementación

```
package juego;

import java.awt.Color;

import entorno.Entorno;

public class Bomba {

private int x;

private int y;

private double ancho;

private double alto;

private boolean exploto;

private int mechaBomb;

private int direccion;

private int velocidadImplosion;

private int tiempoDeExplosion;

public Bomba(int x, int y, double ancho, double alto, int direccion) {

this.x = x;

this.y = y;

this.ancho = ancho;

this.alto = alto;

this.exploto = false;

this.mechaBomb = 180;

this.direccion = direccion;

this.velocidadImplosion = 2;

this.tiempoDeExplosion = 0;

}

public void cuentaRegresiva() {

if(this.mechaBomb>0) {

this.mechaBomb--;
```

```
if(this.mechaBomb==0) {

this.exploto = true;

}

}

}

public void dibujar (Entorno entorno) {

if (this.exploto) {

entorno.dibujarRectangulo(this.x,    this.y,    this.ancho,    this.alto,    0,
Color.YELLOW);

}else{

entorno.dibujarCirculo(this.x, this.y, this.alto, Color.DARK_GRAY);

}

}


public int getXBomb() {

return this.x;

}


public int getYBomb() {

return this.y;

}


public double getAltoBomb() {

return this.alto;

}


public double getAnchoBomb() {
```

```
return this.ancho;
}
public void crecerExplosion() {
if (this.exploto) {
if (this.direccion == 1 || this.direccion == 2) {
if (this.ancho<=160) {
this.ancho+=this.velocidadImplosion;
}
if(this.alto<=45) {
this.alto+=this.velocidadImplosion;
}
}
if (this.direccion == 3 || this.direccion == 4) {
if (this.ancho<=56) {
this.ancho+=this.velocidadImplosion;
}
if(this.alto<=150) {
this.alto+=this.velocidadImplosion;
}
}
}
}
public boolean duracionExp() {
    if (this.exploto) {
        if (this.tiempoDeExplosion < 120) {
            this.tiempoDeExplosion++;
            return false;
```

```
        } else {
            this.tiempoDeExplosion = 0;
            return true;
        }
    }
    return false;
}

}
```

Clase Enemigo

La clase Enemigo representa a los enemigos que aparecen en el juego. Se encarga de almacenar su posición, sus dimensiones, su velocidad y su dirección de movimiento. También permite dibujarlos en pantalla, moverlos en las cuatro direcciones y controlar el intervalo entre las bombas.

Variables de instancia

- x e y: indican la posición del enemigo en la pantalla.
- ancho y alto: representan las dimensiones del enemigo.
- velocidad: determina cuánto se desplaza el enemigo en cada movimiento.
- direccion: indica la dirección de movimiento del enemigo.
- intervaloEntreBombas: controla el tiempo que transcurre entre una bomba y la siguiente.

Métodos

- Enemigo(int x, int y, int ancho, int alto, int velociad, int direccion, int intervaloEntreBombas): es el constructor de la clase. Inicializa la posición, las dimensiones, la velocidad, la dirección y el intervalo entre bombas.
- dibujar(Entorno entorno): dibuja al enemigo como un rectángulo rojo en la pantalla.
- eneMovDer(): mueve al enemigo hacia la derecha, aumentando su coordenada x según su velocidad.
- eneMovIzq(): mueve al enemigo hacia la izquierda, disminuyendo su coordenada x según su velocidad.
- eneMovArr(): mueve al enemigo hacia arriba, disminuyendo su coordenada y según su velocidad.
- eneMovAbaj(): mueve al enemigo hacia abajo, aumentando su coordenada y según su velocidad.
- getXEn(): devuelve la coordenada horizontal del enemigo.
- getYEn(): devuelve la coordenada vertical del enemigo.
- getAnchoEn(): devuelve el ancho del enemigo.
- getAltoEn(): devuelve el alto del enemigo.
- getDirecEn(): devuelve la dirección actual del enemigo.
- getVelocidad(): devuelve la velocidad del enemigo.
- setDirecEn(int direccion): permite modificar la dirección del enemigo.
- bombaTime(): controla el intervalo entre bombas. Mientras el contador sea menor que 600, lo incrementa y devuelve false. Cuando llega al límite, reinicia el contador y devuelve true, indicando que se cumplió el intervalo establecido.

Decisiones de diseño

Código de implementación

```
package juego;

import java.awt.Color;

import entorno.Entorno;

public class Enemigo {

private int x;

private int y;

private int ancho;

private int alto;

private int velocidad;

private int direccion;

private int intervaloEntreBombas;

public Enemigo(int x, int y, int ancho, int alto, int velociad, int direccion,
int intervaloEntreBombas) {

this.x = x;

this.y = y;

this.ancho = ancho;

this.alto = alto;

this.velocidad =  velociad;

this.direccion =  direccion;

this.intervaloEntreBombas = intervaloEntreBombas;

}

public void dibujar(Entorno entorno) {

entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, 0, Color.RED);

}
```

```
public void eneMovDer() {
this.x += this.velocidad;
}
public void eneMovIzq() {
this.x -= this.velocidad;
}
public void eneMovArr() {
this.y -= this.velocidad;
}
public void eneMovAbaj() {
this.y += this.velocidad;
}
public int getXEn() {
return x;
}
public int getYEn() {
return y;
}

public int getAnchoEn() {
        return this.ancho;
    }
    public int getAltoEn() {
        return this.alto;
    }
    public int getDirecEn() {
```

```
    return this.direccion;
  }
  public int getVelocidad() {
    return this.velocidad;
  }
  public void setDirecEn(int direccion) {
    this.direccion = direccion;
  }
  public boolean bombaTime() {
    if (this.intervaloEntreBombas<600) {
          this.intervaloEntreBombas++;
          return false;
    }else {
          this.intervaloEntreBombas = 0;
          return true;
    }
 }
}
```

Clase Juego

La clase Juego es la clase principal del programa. Se encarga de coordinar el funcionamiento general del juego, controlar al repartidor, los enemigos, los disparos, los pedidos, las colisiones y la interfaz. También controla las condiciones de victoria y derrota.

Variables de instancia

- entorno: permite dibujar los elementos del juego, detectar las teclas presionadas y registrar los clics del mouse.
- mapa: representa el mapa donde se desarrolla el juego.
- repartidor: representa al personaje que controla el jugador.
- disparo: guarda el proyectil que dispara el repartidor.
- flecha: indica mediante una flecha el lugar al que debe dirigirse el repartidor.
- interfaz: muestra las estadísticas y el estado del pedido en pantalla.
- enemigos: es un arreglo que contiene los cuatro enemigos del juego.
- juegoTerminado: indica si el juego terminó por victoria o derrota.
- vidas: almacena la cantidad de vidas disponibles. Comienza en 3.
- pedidosEntregados: cuenta la cantidad de pedidos entregados correctamente.
- enemigosEliminados: cuenta la cantidad de enemigos eliminados.
- tiempoInvulnerable: evita que el repartidor pierda varias vidas seguidas al permanecer en contacto con un enemigo.
- estadoPedidoActual: guarda el estado en el que se encuentra el pedido, por ejemplo, "REALIZADO", "ESPERANDO", "EN_CAMINO", "ENTREGADO" o "PERDIDO".
- negocioActual: guarda el negocio donde debe comenzar el pedido.
- casaActual: guarda la casa donde debe entregarse el pedido.
- tiempoActualEspera: cuenta el tiempo de espera antes de generar un nuevo pedido.
- tiempoActualEntrega: cuenta el tiempo disponible para completar la entrega.
- tiempoActualResultado: cuenta cuánto tiempo se muestra el resultado de un pedido antes de comenzar el siguiente.
- TIEMPO_ESPERA_REALIZADO: establece el tiempo de espera para generar un pedido, en 120 ticks.
- TIEMPO_LIMITE_ENTREGA: establece el límite para completar una entrega, en 600 ticks.
- TIEMPO_MOSTRAR_RESULTADO: establece cuánto tiempo se muestra el resultado del pedido, en 120 ticks.

Métodos

- Juego() Es el constructor de la clase. Crea el entorno de 800 por 600 píxeles, inicializa el mapa, el repartidor, la flecha, la interfaz y los cuatro enemigos. Finalmente, inicia el juego mediante entorno.iniciar().
- generarNuevoPedido() Selecciona aleatoriamente un negocio y una casa ubicada en una cuadra diferente. Guarda ambos lugares y cambia el estado del pedido a "ESPERANDO". También muestra la flecha que indica dónde se encuentra el negocio.
- hayColisionConCuadras(double posX, double posY, double pAncho, double pAlto) Comprueba si un objeto, teniendo en cuenta su posición y sus dimensiones, se superpone con alguna cuadra del mapa. Devuelve true si hay colisión y false si no la hay. Se utiliza para evitar que el repartidor atraviese las cuadras y para controlar el movimiento del proyectil.
- tick() Es el método principal del juego y se ejecuta continuamente mientras funciona el entorno. Se encarga de actualizar y dibujar los elementos, controlar el movimiento del repartidor y los enemigos, gestionar los disparos, comprobar las colisiones y actualizar las estadísticas.También controla el ciclo de los pedidos, desde que se generan hasta que se entregan o se pierden. Si el repartidor llega a cinco pedidos entregados conservando al menos una vida, gana. Si se queda sin vidas, pierde.
- main(String[] args) Es el método que inicia la ejecución del programa. Crea un objeto de la clase Juego, cuyo constructor inicializa los elementos y pone en marcha el entorno.

Problemas encontrados y soluciones

Una de las principales dificultades fue integrar las distintas clases y conseguir que las acciones de un elemento produjeran las consecuencias correspondientes en el resto del juego. No alcanzaba

con implementar cada comportamiento por separado, sino que debíamos controlar cómo afectaba al desarrollo de la partida. Para resolverlo, fuimos incorporando las comprobaciones necesarias desde la clase Juego y verificando que cada situación se resolviera correctamente.

Otro problema fue implementar las colisiones sin afectar el funcionamiento de los movimientos. Por un lado, debíamos impedir que el repartidor atravesara las cuadras; por otro, teníamos que detectar los contactos con los enemigos y las bombas para aplicar las consecuencias correspondientes. Para esto, incorporamos métodos que permiten comprobar si los elementos se superponen y utilizar esa información para determinar qué acción debía realizar el juego.

También fue necesario controlar los estados y los tiempos de los pedidos para que las distintas etapas se desarrollaran en el orden esperado. Para organizar este funcionamiento, utilizamos contadores y condiciones que permiten determinar cuándo comienza una etapa, cuándo finaliza y qué sucede si se supera el tiempo disponible.

Por último, tuvimos que coordinar los disparos, los enemigos y las bombas con las demás mecánicas. Por ejemplo, al eliminar a un enemigo, era necesario actualizar el contador correspondiente; cuando una bomba alcanzaba al repartidor, debían aplicarse las consecuencias previstas para esa situación. Resolver estas interacciones requirió relacionar los comportamientos de las distintas clases y comprobar que los cambios se reflejaran correctamente en la partida.

En general, el trabajo sobre la clase Juego nos permitió llevar a la práctica las condiciones establecidas en la consigna y comprender la importancia de coordinar las distintas partes de un programa. A medida que avanzamos, fuimos ajustando la implementación para que los movimientos, las colisiones, los pedidos y las demás mecánicas funcionaran de manera integrada.

Código de implementación

```
package juego;


import java.awt.Color;
import entorno.Entorno;
import entorno.InterfaceJuego;
```

```
import java.util.Random;


public class Juego extends InterfaceJuego
{
private Entorno entorno;
private Mapa mapa;
private Repartidor repartidor;
private Proyectil disparo;
private Flecha flecha;
private Interfaz interfaz;
private boolean juegoTerminado = false;
private Enemigo[] enemigos;
private Bomba[] bombas;
// Variables para estadísticas
private int vidas = 3;
private int pedidosEntregados = 0;
private int enemigosEliminados = 0;
private int pedidosPerdidos = 0;

// Control del ciclo de vida del pedido y temporizadores basados en Ticks
private String estadoPedidoActual = "REALIZADO";
private Negocio negocioActual;
private Casa casaActual;

// Contadores por ticks (asumiendo aprox 60 ticks por segundo)
private int tiempoActualEspera = 0;
private int tiempoActualEntrega = 0;
private int tiempoActualResultado = 0;

// Constantes de tiempo medidas en ticks (ej: 60 ticks = 1 segundo aprox)
private int TIEMPO_ESPERA_REALIZADO = 120; // 2 segundos (60 * 2)
private int TIEMPO_LIMITE_ENTREGA = 600;   // 10 segundos (60 * 10)
private int TIEMPO_MOSTRAR_RESULTADO = 120; // 2 segundos (60 * 2)


Juego()
{
this.entorno = new Entorno(this, "Proyecto para TP", 800, 600);

// Inicializar el mapa, el repartidor y la interfaz
this.mapa = new Mapa();
this.repartidor = new Repartidor(400, 570, 20, 25, 3.0);
this.flecha = new Flecha(0, 0);
this.interfaz = new Interfaz();
this.enemigos = new Enemigo[4];
this.enemigos[0] = new Enemigo(20, 290, 35, 35, 2, 1, 150);
this.enemigos[1] = new Enemigo(780, 430, 35, 35, 2, 2, 450);
this.enemigos[2] = new Enemigo(220, 52, 40, 35, 2, 3, 300);
this.enemigos[3] = new Enemigo(580, 580, 40, 35, 2, 4, 0);
this.bombas = new Bomba[5];

// Inicia el juego!
this.entorno.iniciar();
}

// Método auxiliar para seleccionar un negocio y una casa al azar
private void generarNuevoPedido() {
    Cuadra[] cuadras = this.mapa.getCuadras();
    Random random = new Random(); // Creamos el generador


    // 1. Elegir una cuadra al azar y obtener su negocio
    int indiceCuadraNegocio = random.nextInt(cuadras.length);
    this.negocioActual = cuadras[indiceCuadraNegocio].getNegocio();


    // 2. Elegir una casa de destino distinta
    int indiceCuadraCasa;
    do {
```

```
        indiceCuadraCasa = random.nextInt(cuadras.length);
    } while (indiceCuadraCasa == indiceCuadraNegocio);


    // Obtenemos el arreglo de casas de esa cuadra elegida
    Casa[] casasDeCuadra = cuadras[indiceCuadraCasa].getCasas();


    // Elegimos una casa al azar dentro de ese arreglo
    int indiceCasaAleatoria = random.nextInt(casasDeCuadra.length);
    this.casaActual = casasDeCuadra[indiceCasaAleatoria];


    // 3. Si ambos se obtuvieron correctamente, pasamos al estado ESPERANDO
    if (this.negocioActual != null && this.casaActual != null) {
        this.estadoPedidoActual = "ESPERANDO";
        this.flecha.mostrar(this.negocioActual.getX(), this.negocioActual.getY());
    }
}



// Método de colisión
    private boolean hayColisionConCuadras(double posX, double posY, double pAncho, double
pAlto) {
        Cuadra[] cuadras = this.mapa.getCuadras();


        for (int i = 0; i < cuadras.length; i++) {
            if (cuadras[i] != null) {
                double inicioX = cuadras[i].getX() - (cuadras[i].getAncho() / 2);
                double finX = cuadras[i].getX() + (cuadras[i].getAncho() / 2);
                double inicioY = cuadras[i].getY() - (cuadras[i].getAlto() / 2);
                double finY = cuadras[i].getY() + (cuadras[i].getAlto() / 2);


                if (posX + (pAncho / 2) > inicioX && posX - (pAncho / 2) < finX &&
                    posY + (pAlto / 2) > inicioY && posY - (pAlto / 2) < finY) {
                    return true;
                }
            }
        }
        return false;
    }


    private boolean hayColisionEnemigoRepartidor(Enemigo enemigo) {
        double inicioXEn = enemigo.getXEn() - (enemigo.getAnchoEn()/2);
        double finXEn = enemigo.getXEn() + (enemigo.getAnchoEn()/2);
        double inicioYEn = enemigo.getYEn() - (enemigo.getAltoEn()/2);
        double finYEn = enemigo.getYEn() + (enemigo.getAltoEn()/2);

        double inicioXRep = this.repartidor.getX() - (this.repartidor.getAncho()/2);
        double finXRep = this.repartidor.getX() + (this.repartidor.getAncho()/2);
        double inicioYRep = this.repartidor.getY() - (this.repartidor.getAlto()/2);
        double finYRep = this.repartidor.getY() + (this.repartidor.getAlto()/2);

        if (inicioXEn>finXRep || finXEn<inicioXRep ||
                       inicioYEn>finYRep || finYEn<inicioYRep) {
                return false;
        } else {
                return true;
        }
    }



    private boolean hayColisionEnemigoProyectil(Enemigo enemigo) {
        double inicioXEn = enemigo.getXEn() - (enemigo.getAnchoEn()/2);
        double finXEn = enemigo.getXEn() + (enemigo.getAnchoEn()/2);
        double inicioYEn = enemigo.getYEn() - (enemigo.getAltoEn()/2);
```

```
        double finYEn = enemigo.getYEn() + (enemigo.getAltoEn()/2);

        double inicioXPro = this.disparo.getX() - (this.disparo.getAncho()/2);
        double finXPro = this.disparo.getX() + (this.disparo.getAncho()/2);
        double inicioYPro = this.disparo.getY() - (this.disparo.getAlto()/2);
        double finYPro = this.disparo.getY() + (this.disparo.getAlto()/2);

        if (inicioXEn>finXPro || finXEn<inicioXPro ||
                        inicioYEn>finYPro || finYEn<inicioYPro) {
                return false;
        } else {
                return true;
        }
    }


    private void generarNuevoEnemigo(int i) {
        if(i==0) {
                this.enemigos[i] = new Enemigo(20, 290, 35, 35, 2, 1, 150);
        }else if(i==1) {
                this.enemigos[i] = new Enemigo(780, 430, 35, 35, 2, 2, 450);
        }else if(i==2) {
                this.enemigos[i] = new Enemigo(220, 52, 40, 35, 2, 3, 300);
        }else if(i==3) {
                this.enemigos[i] = new Enemigo(580, 580, 40, 35, 2, 4, 0);
        }
    }


    private boolean hayColisionBombaRepartidor(Bomba bomba){
        double inicioXBomb = bomba.getXBomb() - (bomba.getAnchoBomb()/2);
        double finXBomb = bomba.getXBomb() + (bomba.getAnchoBomb()/2);
        double inicioYBomb = bomba.getYBomb() - (bomba.getAltoBomb()/2);
        double finYBomb = bomba.getYBomb() + (bomba.getAltoBomb()/2);

        double inicioXRep = this.repartidor.getX() - (this.repartidor.getAncho()/2);
        double finXRep = this.repartidor.getX() + (this.repartidor.getAncho()/2);
        double inicioYRep = this.repartidor.getY() - (this.repartidor.getAlto()/2);
        double finYRep = this.repartidor.getY() + (this.repartidor.getAlto()/2);

        if (inicioXBomb>finXRep || finXBomb<inicioXRep ||
                        inicioYBomb>finYRep || finYBomb<inicioYRep) {
                return false;
        } else {
                return true;
        }
    }


    private void dropBomb(Enemigo enemigo) {
        for (int i=0; i<this.bombas.length; i++) {
                if(this.bombas[i]==null) {
                        this.bombas[i]= new Bomba(enemigo.getXEn(), enemigo.getYEn(), 30, 30,
enemigo.getDirecEn());
                        return;
                }
        }
    }


public void tick()
{
// 1. Dibujar el mapa
this.mapa.dibujar(entorno);

// 2. MÁQUINA DE ESTADOS DEL PEDIDO

// Estado A: REALIZADO -> Espera un tiempo antes de generar el pedido
if (this.estadoPedidoActual.equals("REALIZADO")) {
this.tiempoActualEspera++;
```

```
if (this.tiempoActualEspera >= TIEMPO_ESPERA_REALIZADO) {
generarNuevoPedido();
this.tiempoActualEspera = 0;
}
}

// Estado B: EN_CAMINO
else if (this.estadoPedidoActual.equals("EN_CAMINO")) {
this.tiempoActualEntrega++;

if (this.tiempoActualEntrega >= TIEMPO_LIMITE_ENTREGA) {
// ¡Se agotó el tiempo! Pedido perdido
this.estadoPedidoActual = "PERDIDO";
this.flecha.ocultar();
this.negocioActual = null; // Limpiamos referencias
this.casaActual = null;
this.pedidosPerdidos++;

this.tiempoActualResultado = 0;
}
}

// Estado C: ENTREGADO / PERDIDO -> Muestra el resultado durante unos ticks y vuelve a
REALIZADO
else if (this.estadoPedidoActual.equals("ENTREGADO") ||
this.estadoPedidoActual.equals("PERDIDO")) {
this.tiempoActualResultado++;

if (this.tiempoActualResultado >= TIEMPO_MOSTRAR_RESULTADO) {
this.estadoPedidoActual = "REALIZADO";
this.tiempoActualEspera = 0;
}
}

if (this.vidas > 0 && this.pedidosEntregados >= 5) {
    this.juegoTerminado = true;
}
if(this.vidas == 0 || this.pedidosPerdidos >= 5) {
this.juegoTerminado = true;
if (this.juegoTerminado) {
        this.mapa.dibujar(entorno); // Mantiene el mapa de fondo visible
        entorno.dibujarRectangulo(400, 300, 350, 150, 0, Color.black);
        entorno.cambiarFont("Arial", 30, Color.white);
        entorno.escribirTexto("¡PERDISTE EL JUEGO!", 270, 310);
        return; // Sale del tick y frena cualquier movimiento o actualización
}

}

// Si el juego ya terminó, mostramos el cartel y detenemos todo acá
    if (this.juegoTerminado) {
        this.mapa.dibujar(entorno); // Mantiene el mapa de fondo visible
        entorno.dibujarRectangulo(400, 300, 350, 150, 0, Color.black);
        entorno.cambiarFont("Arial", 30, Color.white);
        entorno.escribirTexto("¡GANASTE EL JUEGO!", 270, 310);
        return; // Sale del tick y frena cualquier movimiento o actualización
}

 // Control de interacción con el Clic Derecho
    if (this.entorno.sePresionoBoton(this.entorno.BOTON_DERECHO)) {
        double mX = entorno.mouseX();
        double mY = entorno.mouseY();


        // Si el pedido está ESPERANDO, chequear cercanía del repartidor Y clic sobre el
negocio
        if (this.estadoPedidoActual.equals("ESPERANDO")) {
```

```
            double distanciaNegocio = Math.hypot(this.repartidor.getX() -
this.negocioActual.getX(),
                                                 this.repartidor.getY() -
this.negocioActual.getY());


            if (distanciaNegocio < 60 && this.negocioActual.estaAdentro(mX, mY)) {
                this.estadoPedidoActual = "EN_CAMINO";
                this.flecha.ocultar();
                this.flecha.mostrar(this.casaActual.getX(), this.casaActual.getY());
                this.tiempoActualEntrega = 0;
            }
        }


        // Si el pedido está EN_CAMINO, chequear cercanía del repartidor Y clic sobre la casa
        else if (this.estadoPedidoActual.equals("EN_CAMINO")) {
            double distanciaCasa = Math.hypot(this.repartidor.getX() - this.casaActual.getX(),
                                              this.repartidor.getY() - this.casaActual.getY());


            if (distanciaCasa < 60 && this.casaActual.estaAdentro(mX, mY)) {
                this.pedidosEntregados++;
                this.flecha.ocultar();
                this.negocioActual = null;
                this.casaActual = null;
                this.estadoPedidoActual = "ENTREGADO";
                this.tiempoActualResultado = 0;
            }
        }
    }

// Obtenemos la posición y dimensiones dinámicamente
double xActual = this.repartidor.getX();
double yActual = this.repartidor.getY();
double anchoRepartidor = this.repartidor.getAncho();
double altoRepartidor = this.repartidor.getAlto();
double velocidad = 3.0;

// 3. Control de movimiento
if (this.entorno.estaPresionada('d') ||
this.entorno.estaPresionada(this.entorno.TECLA_DERECHA)) {
if (xActual + velocidad < 790 && !this.hayColisionConCuadras(xActual + velocidad, yActual,
anchoRepartidor, altoRepartidor)) {
this.repartidor.moverDerecha();
}
}
if (this.entorno.estaPresionada('a') ||
this.entorno.estaPresionada(this.entorno.TECLA_IZQUIERDA)) {
if (xActual - velocidad > 18 && !this.hayColisionConCuadras(xActual - velocidad, yActual,
anchoRepartidor, altoRepartidor)) {
this.repartidor.moverIzquierda();
}
}
if (this.entorno.estaPresionada('w') || this.entorno.estaPresionada(this.entorno.TECLA_ARRIBA))
{
if (yActual - velocidad > 50 && !this.hayColisionConCuadras(xActual, yActual - velocidad,
anchoRepartidor, altoRepartidor)) {
this.repartidor.moverArriba();
}
}
if (this.entorno.estaPresionada('s') || this.entorno.estaPresionada(this.entorno.TECLA_ABAJO))
{
if (yActual + velocidad < 590 && !this.hayColisionConCuadras(xActual, yActual + velocidad,
anchoRepartidor, altoRepartidor)) {
this.repartidor.moverAbajo();
}
}

// 4. Dibujar al repartidor y la flecha
```

```
this.repartidor.dibujar(this.entorno);
this.flecha.dibujar(this.entorno);

// 5. Control de disparo
if (entorno.sePresionoBoton(entorno.BOTON_IZQUIERDO) && this.disparo == null) {
    this.disparo = this.repartidor.disparar();
}


// 6. Mover, dibujar y controlar el disparo
if (this.disparo != null) {
this.disparo.mover();
this.disparo.dibujar(entorno);


if (this.disparo.getX() < 0 || this.disparo.getX() > 800 ||
this.disparo.getY() < 0 || this.disparo.getY() > 600 ||
this.hayColisionConCuadras(this.disparo.getX(), this.disparo.getY(), this.disparo.getAncho(),
this.disparo.getAlto())) {

this.disparo = null;
}
}

for (int i= 0; i<enemigos.length;i++) {
if(this.enemigos[i]==null) {
generarNuevoEnemigo(i);
}
if (this.enemigos[i]!=null) {
this.enemigos[i].dibujar(entorno);

if (this.enemigos[i].getDirecEn() == 1) {
    if (this.enemigos[i].getXEn() + this.enemigos[i].getVelocidad() < 790) {
        this.enemigos[i].eneMovDer();
    } else {
        this.enemigos[i].setDirecEn(2);
    }
}

if (this.enemigos[i].getDirecEn() == 2) {
    if (this.enemigos[i].getXEn() - this.enemigos[i].getVelocidad() > 16) {
        this.enemigos[i].eneMovIzq();
    } else {
        this.enemigos[i].setDirecEn(1);
    }
}

if (this.enemigos[i].getDirecEn() == 3) {
    if (this.enemigos[i].getYEn() - this.enemigos[i].getVelocidad() > 50) {
        this.enemigos[i].eneMovArr();
    } else {
        this.enemigos[i].setDirecEn(4);
    }
}

if (this.enemigos[i].getDirecEn() == 4) {
    if (this.enemigos[i].getYEn() + this.enemigos[i].getVelocidad() < 590) {
        this.enemigos[i].eneMovAbaj();
    } else {
        this.enemigos[i].setDirecEn(3);
    }
}
if (hayColisionEnemigoRepartidor(this.enemigos[i])) {
this.vidas--;
this.repartidor.respawnRep(400, 570);
    if (this.estadoPedidoActual.equals("EN_CAMINO")) {
        this.estadoPedidoActual = "PERDIDO";
        this.flecha.ocultar();
        this.negocioActual = null;
```

```
        this.casaActual = null;
        this.pedidosPerdidos++;
        this.tiempoActualResultado = 0;
    }

}
if (this.disparo != null) {
if (hayColisionEnemigoProyectil(this.enemigos[i])) {
enemigos[i]=null;
this.disparo=null;
this.enemigosEliminados++;
}
}

if (enemigos[i] != null) {
    if (enemigos[i].bombaTime()) {
        dropBomb(enemigos[i]);
    }
}
}
}

for (int i=0;i<bombas.length;i++) {
if (bombas[i]!=null) {
this.bombas[i].cuentaRegresiva();
this.bombas[i].crecerExplosion();
this.bombas[i].dibujar(entorno);
if (hayColisionBombaRepartidor(this.bombas[i])) {
    this.vidas = 0;
}
if (this.bombas[i].duracionExp()) {
this.bombas[i] = null;
}

}
}


// 7. Dibujar la interfaz
this.interfaz.dibujar(entorno, vidas, pedidosEntregados, pedidosPerdidos, enemigosEliminados,
estadoPedidoActual);
}


@SuppressWarnings("unused")
public static void main(String[] args)
{
Juego juego = new Juego();
}
}
```

Conclusión

El desarrollo de *Delivery Rush* fue una experiencia que nos permitió poner en práctica los conocimientos adquiridos durante la cursada y, al mismo tiempo, aprender de las dificultades que fueron surgiendo a lo largo del trabajo. Desarrollar un juego desde cero implicó mucho más que escribir código: tuvimos que pensar cómo organizarlo, cómo hacer que las distintas clases se relacionaran entre sí y cómo resolver los problemas que aparecían a medida que avanzábamos.

La división y delegación de tareas nos permitió organizarnos mejor y avanzar con diferentes partes del proyecto, pero también nos hizo ver la importancia de mantener una comunicación constante para que todo pudiera integrarse correctamente. Fue necesario compartir los cambios realizados, revisar el trabajo de los demás y ponernos de acuerdo cuando surgían distintas formas de resolver un mismo problema.

También fue importante tener en cuenta las recomendaciones y correcciones de los profesores. Muchas veces, revisar una idea o modificar algo que ya habíamos hecho nos permitió encontrar mejores soluciones y comprender aspectos del código que antes nos resultaban difíciles.

En cuanto al juego, trabajar con los movimientos del repartidor, las colisiones, los pedidos y la interacción entre los distintos elementos nos ayudó a comprender mejor cómo aplicar la programación orientada a objetos en un proyecto concreto. Cada funcionalidad requirió pruebas y ajustes, y ver cómo las diferentes partes comenzaban a funcionar en conjunto fue una parte importante de la experiencia.

En conclusión, este trabajo no solo nos permitió reforzar los contenidos de la materia, sino también aprender a organizarnos, compartir ideas, aceptar correcciones y buscar soluciones en conjunto.

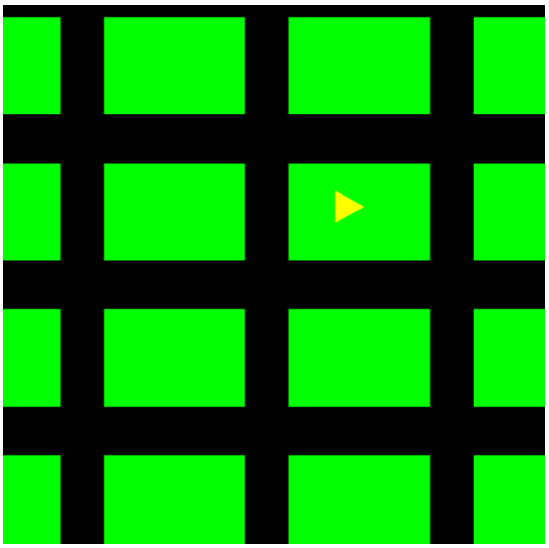
Anexo:

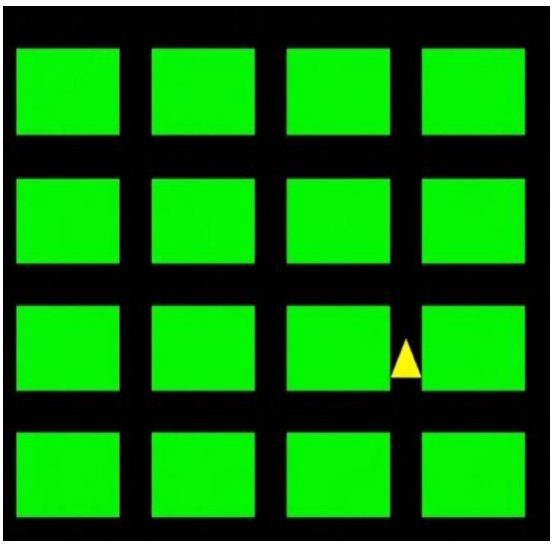
En este anexo se presentan imágenes que muestran las distintas etapas del desarrollo de *Delivery Rush*. A través de ellas, se puede observar cómo fue evolucionando el juego a medida que incorporamos sus diferentes elementos y funcionalidades.

Las capturas permiten visualizar los avances realizados durante el proceso, desde las primeras versiones hasta la incorporación de los personajes, el mapa y las mecánicas principales del juego.

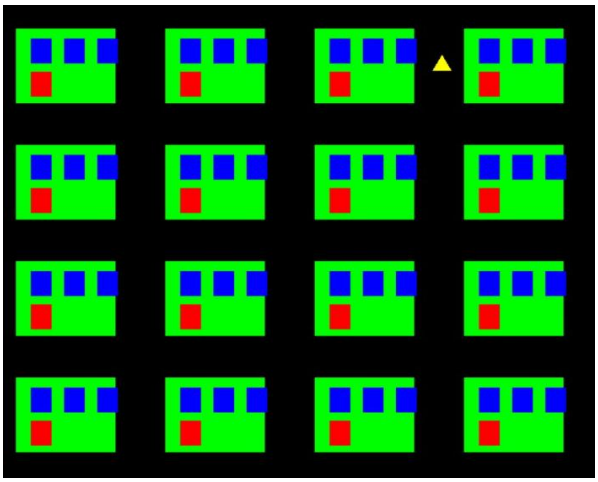
Este registro nos permite dejar constancia del trabajo realizado y observar cómo, a partir de las pruebas, las correcciones y los cambios implementados, fuimos construyendo la versión final del proyecto.

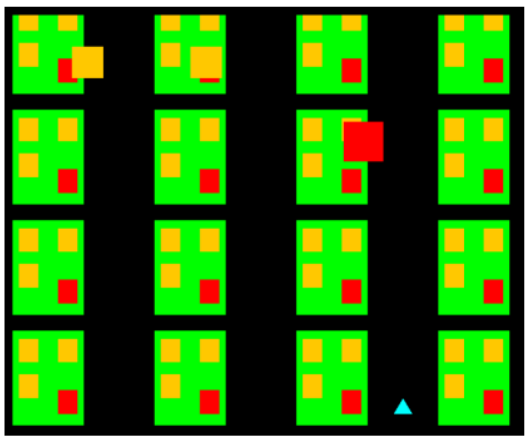
Repartidor traspasando las cuadras



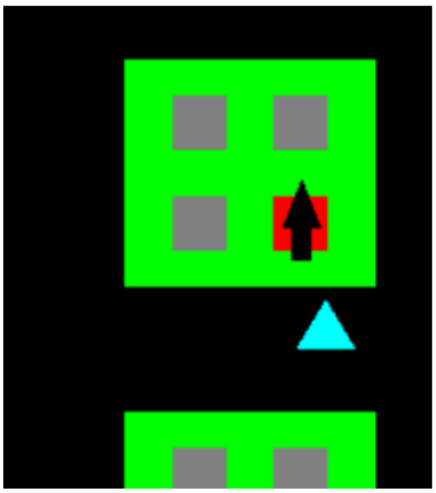


Negocios mal dimensionados





Flecha



Juego final

